

Operating Systems Lab (CL2006)

Date: May, 22, 2026

Instructors: All

Sections: All

Final Exam (Lab)

Total Time (H:M): 2:00

Total Marks: 50

Total Questions: 2

Semester: Spring-2026

Roll No	Section	Student Signature
---------	---------	-------------------

Submission paths:

\\Exam\Final Exam\Operating Systems\[Your Section]\Submissions

Instructions:

- Open NS3_1 virtual machine (placed in C or D drive, password is 123456, alternatively you can use NS2 vm.
- Check if you are able to copy paste your files from Ubuntu to Windows, if not inform the invigilator at start [You can also copy code from ubuntu and create a new (.txt) file in windows].
- Submit a zip folder with your code and screenshots named as your rollNumber.

Attempt all the questions.

Q. No 1: Multithreaded Word Count Server with FIFO [40]

Build a 3 program system which will include: wrapper, client, and server.

1. The wrapper process creates a named pipe (FIFO) and uses fork() + exec() to launch the client and the server as separate processes.
2. The client reads lines from a text file and sends them through the FIFO.
3. The server receives each line, spawns a dedicated worker thread for it, and each worker thread counts the words in its line. All threads write their results into a shared array protected by a mutex. After all threads finish, the server prints the results in order.

Detailed explanation of each part is given below:

1. wrapper.c [12]
 - a. Create a named pipe with permissions 0666 [02]
 - b. Create a child process
 - i. Handle errors if child process creation failed [01]
 - ii. In child launch ./client using exec system call [03]
 - c. In parent process launch ./server using exec [03]
 - i. Format child process id to string and pass it to ./server via command line arguments [03]

Spring 2026

2. client.c [10]
 - a. Open input.txt for reading. Exit with an error message if the file cannot be opened. [02]
 - b. Open FIFO for writing (write only mode) [01]
 - c. Read input.txt line by line and for every line write the entire line (including the newline character) to the FIFO [05]
 - d. Close FIFO and file [02]
3. server.c [15]
 - a. Declare a global struct array Result results[1024] where Result has fields int line_index and int word_count. Declare a global int result_count = 0 and mutex declared and initialised correctly [03]
 - b. Declare a thread argument struct with fields char line[1024] and int index, you can create an array of this so that each index represents a separate thread. [03]
 - c. Write a worker function that will receive a line and it will count the words (in line) and write the result into a shared array. [do proper synchronization]. [06]
 - d. Join the thread and print the final result [03]
4. Output Screenshot submission [03]

Q. No 2: Bank Account Synchronization Using Shared Memory [10]

Write a program to simulate a bank account system where multiple processes safely update a shared account balance using shared memory and semaphores.

1. Create a shared memory segment using shm_open() and map it using mmap(). The shared memory should store a single integer representing the account balance. [04]
2. Initialize the account balance to 1000.
3. Create two child processes using fork():
 - Process 1 (Deposit):
 - Adds a fixed amount (e.g., +500) to the balance.
 - Must use semaphore locking before updating shared memory. [02]
 - Process 2 (Withdraw):
 - Subtracts a fixed amount (e.g., -300) from the balance.
 - Must also use semaphore locking. [02]
4. The parent process should print the final account balance after both processes finish. [01]
5. Output Screenshot [01]